Lesson 04 · Collections, LINQ & Functional C#

Java Streams, Kotlin collections and TypeScript array methods, re-mapped onto .NET collections, LINQ, delegates and expression trees

Contents

1 · Why collections and LINQ decide how .NET code reads 2
- 1.1 · The numbers to leave this lesson with 2

2 · The collection family — what to accept, store and return 2
- 2.1 · Two interface families — what each type promises 3
- 2.2 · Collection expressions — one literal for every target 3
- 2.3 · Read-only is not immutable 3
- 2.4 · The collection family, mapped from your stack 4
- 2.5 · Choosing the type at an API boundary 5

3 · Deferred execution — a query is a recipe, not a result 5
- 3.1 · What runs when — one query, four calls 6
- 3.2 · Reusing a query — Java Streams vs LINQ 6
- 3.3 · Predicate calls for the same three reads 6
- 3.4 · Deferred means late: captured variables and changing sources 7
- 3.5 · Writing a lazy sequence — Kotlin sequence {} vs C# yield return 7

4 · The operator map — Streams, Kotlin and TypeScript in LINQ 7
- 4.1 · Concept map — pipelines you know → LINQ 7
- 4.2 · When each feature arrived — dating the LINQ in a codebase 9
- 4.3 · AggregateBy — a fold per key without building groups 9
- 4.4 · A key that is not there — lookup vs dictionary 9

5 · Functional C# — delegates, closures and immutable records 9
- 5.1 · Rating rules as a list of functions, folded with Aggregate 10
- 5.2 · Closures in loops — TypeScript let vs C# for 11
- 5.3 · Delegates are nominal; with copies are shallow 11

6 · Quote analytics — the dashboard habit in LINQ 11
- 6.1 · Premium bands — a switch expression inside CountBy 11
- 6.2 · Issuance — LeftJoin, and the query-syntax left join it replaces 12
- 6.3 · What the dashboard prints 12
- 6.4 · Conversion rate by coverage class 13
- 6.5 · Quotes per premium band 13

7 · Expression trees — code as data, and a DynamoDB query generator 13
- 7.1 · One lambda, two destinations 14
- 7.2 · Func<T,bool> or Expression<Func<T,bool>>? 14
- 7.3 · Func vs Expression, a span surprise, and a generated filter 14
- 7.4 · What the expression experiments print 15
- 7.5 · The translator — pattern matching is the visitor 15
- 7.6 · What the generator translates — each row is a passing test 16
- 7.7 · Testing the generator, including what it refuses 16

8 · Visual Basic LINQ — the richer query syntax 16
- 8.1 · Conversion by class — C# methods vs VB Group By … Into 17
- 8.2 · Top makes — CountBy and Index vs a five-line VB query 17
- 8.3 · Aggregate … Into — a clause C# does not have 17
- 8.4 · Distinct, Skip While and Take While — clauses C# spells as methods 18
- 8.5 · What the VB console prints 18
- 8.6 · Which operators have query syntax 18

9 · Hands-on — build, test and run the lesson-04 samples 18
- 9.1 · Commands 19
- 9.2 · What the tests report 19
- 9.3 · Where the lesson-04 code lives 19

Page numbers are resolved from the rendered PDF. Code panels are cut from the compiled samples.

1 · Why collections and LINQ decide how .NET code reads

Most business C# is collection code: load some records, filter, group, aggregate, map to a response — and LINQ is the vocabulary every reviewer expects you to read at a glance. You already think in pipelines: Java Streams and Collectors in Spring services, Kotlin collection operators, TypeScript array methods in NestJS and React, and a few hundred dashboard queries for a BI tool. Most operators map one-to-one. This lesson spends its words on the ones that do not.

Three things do not transfer, and they are where production bugs come from. A LINQ query over `IEnumerable<T>` is re-runnable and lazy, so it silently runs again every time it is enumerated. A lambda can compile to executable code *or* to a data structure a provider translates to SQL or a DynamoDB expression, depending only on the parameter type. And collection interfaces make promises — read-only, immutable, frozen — that are easy to confuse.

You will build the tool you already built once in another language: a query generator. The samples turn a C# predicate into a DynamoDB filter expression with `#name` and `:value` placeholders, run MotorQuote dashboard queries over a deterministic quote book, and repeat the dashboard in Visual Basic query syntax — the form you will meet in older estates.

1.1 · The numbers to leave this lesson with

ONE QUERY, THREE PASSES	NEW IN .NET 9 LINQ	VB-ONLY QUERY CLAUSES	DYNAMODB IN LIST	LESSON 04 SAMPLES
482 calls	**3**	**6**	**≤ 100**	**919 lines**
predicate calls over 240 quotes · measured	CountBy · AggregateBy · Index · verified	no C# query keyword · verified	values per IN operator · verified	7 projects · 43 tests · measured

KEY: • Same idea • Renamed • Different • Trap • Verified • Estimate • Measured C# Java/Kotlin architecture

i Before you start

1. **You need lesson 02 (C# Language Essentials) and lesson 03 (Types, OOP & Generics).** Records, `with`, generics and extension methods are used here without explanation; LINQ operators *are* extension methods on `IEnumerable<T>`.
2. **You will build seven projects.** `L04.QuoteData` (MotorQuote records, a deterministic 240-quote book, dashboard queries, rating rules as functions), `L04.QueryGen` (expression tree → DynamoDB filter), two C# consoles (`L04.Collections`, `L04.QuoteAnalytics`), a VB console (`L04.VbLinq`) and two xUnit projects. The rating rules are the track's one illustrative tariff — the same base rates, loadings, no-claim ladder and rounding every lesson that prices a MotorQuote quote uses, so the figures here and in lesson 02 (C# Language Essentials) agree.
3. **The dataset is an analytics row, not the lesson 03 (Types, OOP & Generics) entity.** Every query here is a read, so `Quote` is a `record` and `QuoteId` / `PolicyNumber` are flattened to `string` — the shape a dashboard row and a DynamoDB item actually have. lesson 03 (Types, OOP & Generics) owns the type-kind decision itself (an entity with identity is a `sealed class`; an id is a `readonly record struct`), and this lesson does not overturn it.
4. **Not here:** `IAsyncEnumerable<T>` is lesson 05 (Async, Tasks & Concurrency); how EF Core translates `IQueryable` to SQL is lesson 09 (Data Access with EF Core); VB language semantics in depth are lesson 06 (VB.NET for Legacy Estates).
5. Facts that change with releases are marked • Verified and link to the source; figures computed from the samples are • Measured ; Java, Kotlin and TypeScript panels are for comparison and are not compiled.

2 · The collection family — what to accept, store and return

Pick a collection type by the promise it makes to the caller, not by the data it holds. `IEnumerable<T>` promises only that you can iterate — perhaps lazily, perhaps expensively. `IReadOnlyList<T>` promises a count and an indexer over data that already exists. `ImmutableArray<T>` and `FrozenDictionary<TKey,TValue>` promise that nothing will change. Kotlin's read-only `List<T>` is the `IReadOnlyList<T>` idea; Java has no read-only interface, only wrappers that throw.

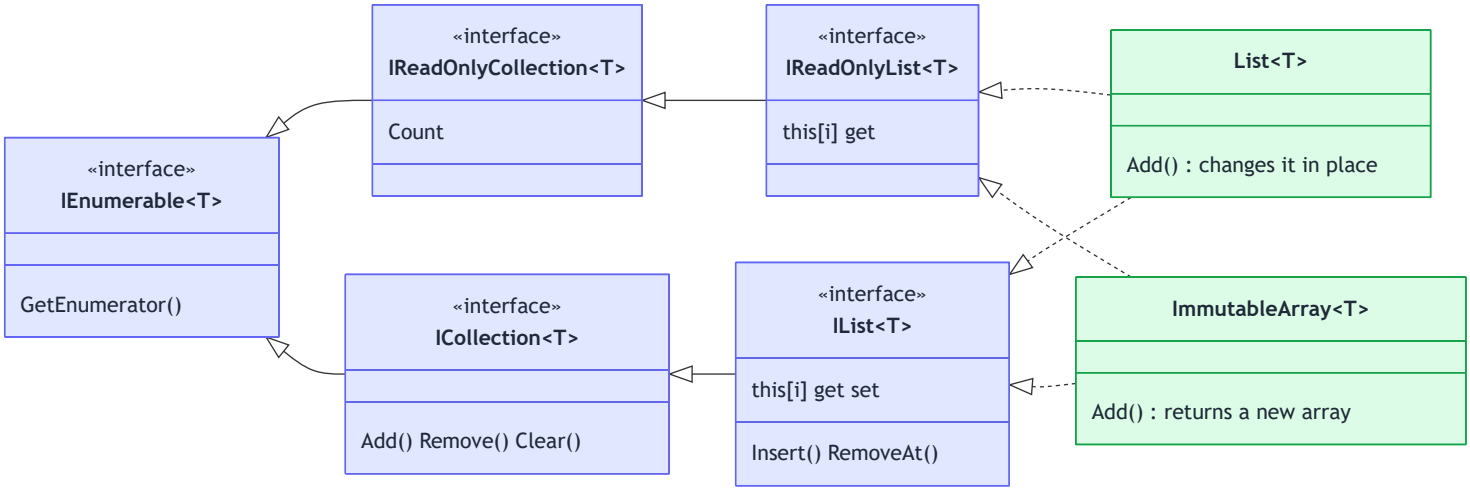
Read-only is a view; immutable is a guarantee. In 2.3 one list is exposed four ways and then grows. The interface view and the `AsReadOnly()` wrapper see the new item; the immutable and frozen copies do not. A caller holding the `IReadOnlyList<T>` view can even cast it back to `List<T>` and remove an item — it compiles. The `AsReadOnly()` wrapper is a different type, so that cast fails; it still cannot stop the owner's changes.

The two interface families are separate. `IList<T>` extends `ICollection<T>` and `IEnumerable<T>` only — not `IReadOnlyList<T>` (IList<T>). Concrete types such as `List<T>`, arrays and `ImmutableArray<T>` implement both, so a method that asks for `IReadOnlyList<T>` accepts all of them.

Collection expressions are one literal syntax for all of them. C# 12 (November 2023, The history of C#) added `[a, b, .. rest]`, which converts to arrays, spans, `List<T>`, immutable types and the collection interfaces. For an interface target the

compiler chooses the concrete type (Collection expressions): the sample's `IReadOnlyList<string>` is *not* a `List<string>`.

2.1 · Two interface families — what each type promises



indigo = interfaces · green = concrete types · solid arrow = extends · dotted arrow = implements · IList<T> and IReadOnlyList<T> are unrelated · arrays and AsReadOnly() wrappers implement both families too

2.2 · Collection expressions — one literal for every target

C# — samples/L04.Collections/Program.cs · #region collection-expressions

```
int[] claimFreeYears = [0, 1, 2, 3, 4, 5];
List<string> makes = ["Toyota", "Honda"];
string[] shortlist = [.. makes, "Isuzu", .. makes.Take(1)];
IReadOnlyList<string> channels = ["SMS", "Email", "LINE"];

Console.WriteLine($"  shortlist  {string.Join(", ", shortlist)}");
Console.WriteLine($"  years      {claimFreeYears.Length} elements");
Console.WriteLine($"  channels is a List<string>? {channels is List<string>}");
Console.WriteLine($"  channels type {channels.GetType().Name}");
```

The spread `..` inlines any enumerable, including a LINQ query. The output is `Toyota, Honda, Isuzu, Toyota` and `6 elements`. The last two lines print `False` and `<>z__ReadOnlyArray`1`: for an `IReadOnlyList<T>` target this compiler synthesised its own read-only type, so a caller cannot cast it to `List<T>` and add. The type name is a compiler detail, not a contract. Unlike a LINQ query, a collection expression is always evaluated into memory immediately (Collection expressions).

2.3 · Read-only is not immutable

C# · four views — Program.cs · #region readonly-views

```
var classes = new List<string> { "Class1", "Class3" };
IReadOnlyList<string> view = classes; // same object
var wrapper = classes.AsReadOnly();   // live wrapper
var snapshot = classes.ToImmutableArray(); // copy
var frozen = classes.ToFrozenSet();   // copy, fast reads

classes.Add("Class2Plus");
Console.WriteLine($"  view      {view.Count}");
Console.WriteLine($"  wrapper   {wrapper.Count}");
Console.WriteLine($"  snapshot  {snapshot.Length}");
Console.WriteLine($"  frozen    {frozen.Count}");

((List<string>)view).Remove("Class1"); // compiles!
Console.WriteLine($"  after cast + Remove  {view.Count}");
```

Output · part 2 — L04.Collections, captured

```
2. read-only is not immutable
  view      3
  wrapper   3
  snapshot  2
  frozen    2
  after cast + Remove  2
```

Only the copies are safe from the owner. `view` is the same object as `classes`, and `AsReadOnly()` wraps it live, so both report 3 after the `Add`. `ToImmutableArray()` and `ToFrozenSet()` copied, so they report 2. The last line shows why "read-only" is a courtesy: the cast back to `List<string>` removed an item through the view. The same cast on `wrapper` would throw `InvalidCastException`, but the wrapper still shows every later change. Return `IReadOnlyList<T>` from APIs, but hand out an immutable copy when the caller must not see — or cause — changes.

2.4 · The collection family, mapped from your stack

types you will review

1 List<T> — the default mutable list C# Java/Kotlin • Same idea

A growable array: O(1) index, amortised O(1) append.

JAVA	`ArrayList<E>`
KOTLIN	`MutableList` / `ArrayList`
TS	`Array<T>`
WATCH	Adding inside `foreach` throws `InvalidOperationException` (3.4) — Java's `ConcurrentModificationException`

2 Dictionary<TKey,TValue> and HashSet<T> C# • Trap

Hash map and hash set; iteration order is not part of the contract.

JAVA	`HashMap` · `HashSet`
KOTLIN	`mutableMapOf` · `mutableSetOf`
TRAP	`dict[key]` on a missing key **throws** `KeyNotFoundException` (4.4); Kotlin `map[key]` and Java `get` return null — use `TryGetValue`
ORDERED	`OrderedDictionary<TKey,TValue>` is generic since .NET 9 (What's new in .NET 9 libraries)

3 Arrays, Span<T> and ReadOnlySpan<T> C# • Different

Fixed-length arrays, and a stack-only window over an array, a string or stack memory.

JAVA	`T[]` · a `ByteBuffer` slice
SPAN	A `ref struct`: it cannot be a class field, be boxed or be captured by a lambda (ref struct types)
USE WHEN	Parsing and hot loops; not in domain models or LINQ pipelines
C# 14	Arrays convert to spans implicitly — which changed LINQ overload binding (7.4)

2.4 · The collection family (continued)

types you will review

4 Immutable collections C# • Different

`ImmutableArray<T>`, `ImmutableList<T>`, `ImmutableDictionary` — every change returns a new instance.

JAVA	`List.copyOf` and Guava `ImmutableList` are unmodifiable: add throws, where `ImmutableList.Add` returns a new list
KOTLIN	`kotlinx.collections.immutable` persistent collections: add returns a new one
USE WHEN	A snapshot shared across requests or threads
TRAP	A record compares an `ImmutableArray<T>` by array reference, not by elements (5)
SAMPLE	`ToImmutableArray()` kept 2 items after the source list grew (2.3)

5 Frozen collections (.NET 8) C# architecture • Different

`FrozenDictionary` and `FrozenSet`: pay once at creation, read faster for the life of the process.

JAVA	`Map.copyOf` · Guava `ImmutableMap` — immutable snapshots
USE WHEN	Start-up lookup tables read on every request: base rates by class, partner endpoints, feature flags
AVOID	Anything rebuilt per request — creation is the expensive part
SAMPLE	`baseRates` in `L04.Collections` is a `FrozenDictionary<CoverageClass, decimal>` · What's new in .NET 8

6 Read-only interfaces and wrappers C# Java/Kotlin • Trap

`IReadOnlyList<T>`, `AsReadOnly()`, `ReadOnlySet<T>` (.NET 9) — views over data someone else can still change.

KOTLIN	`List<T>` over a `MutableList` — the same view semantics
JAVA	`Collections.unmodifiableList` — a view that throws on write
TRAP	A cast of the `IReadOnlyList<T>` view back to `List<T>` compiles and mutates (2.3); the `AsReadOnly()` wrapper refuses that cast
TRAP	`IList<T>` does not extend `IReadOnlyList<T>` (IList<T>)

2.5 · Choosing the type at an API boundary

Position	Use	Why	Habit it replaces
Parameter you only iterate	`IEnumerable<T>`	Accepts arrays, lists and queries; enumerate it **once**	`Iterable<T>`
Parameter you count or index	`IReadOnlyList<T>`	No defensive copy, no second enumeration	Kotlin `List<T>`
Result of a query method	`IReadOnlyList<T>`, materialised	The caller cannot re-run your query by accident (3.2)	`stream().toList()`
A lazy stream you produce	`IEnumerable<T>` iterator; `IAsyncEnumerable<T>` for I/O — lesson 05 (Async, Tasks & Concurrency)	Say it is lazy in the name or the doc comment	Kotlin `Sequence` · `Flow`
Snapshot shared across requests	`ImmutableArray<T>`, `ImmutableDictionary`	Changes return a new instance	Guava immutable collections
Built once, read on every request	`FrozenDictionary`, `FrozenSet`	Read-optimised; creation costs more	a `static final Map`
Private working state	`List<T>`, `Dictionary<TKey,TValue>`	Fastest to fill; never exposed	`ArrayList`, `HashMap`
Hot loop over a buffer	`Span<T>`, `ReadOnlySpan<T>`	No allocation; stack-only	`ByteBuffer`

3 · Deferred execution — a query is a recipe, not a result

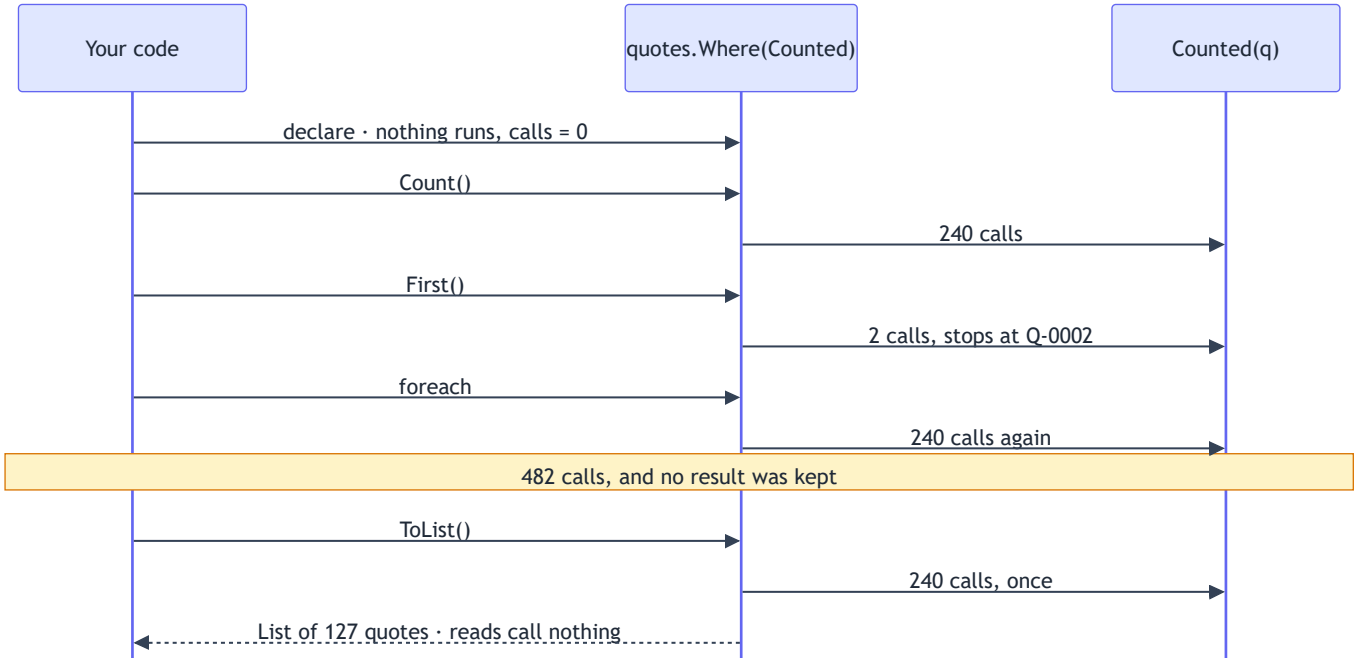
A LINQ query over `IEnumerable<T>` runs every time you enumerate it, and nothing stops you enumerating it twice. A Java `Stream` should be operated on only once and may throw `IllegalStateException` when reused (java.util.stream.Stream). Kotlin sequences can be iterated many times (Kotlin sequences), and LINQ follows Kotlin: reuse is legal, silent and re-executes every step. In the sample a `Count()`, a `First()` and a `foreach` over one query called its predicate **482** times for 240 quotes; `ToList()` once, then three reads, called it 240 times.

Materialise at the boundary. `ToList`, `ToArray`, `ToDictionary` and `ToLookup` execute immediately and keep the results; `Count`, `First` and `Sum` execute immediately and keep nothing; `Where` and `Select` are deferred and streaming; `GroupBy` and `OrderBy` are deferred but read the whole source each time they run (Introduction to LINQ queries). A repository or service method that returns a deferred query hands its cost — and, with EF Core, its open context — to every caller; lesson 09 (Data Access with EF Core) covers that case.

Deferred also means late. A query reads captured variables and the source collection when it is enumerated, not when it is written: raising a threshold after declaring the query changed its count from 123 to 67. Changing a `List<T>` while a `foreach` walks it throws `InvalidOperationException`, as Java throws `ConcurrentModificationException`.

`yield return` is how you write your own lazy sequence. The compiler rewrites the method into a state machine whose body does not start until the first `MoveNext` — the output prints *iterator created* before *(iterator body starts)*. Kotlin's `sequence { yield(x) }` is the same idea with a different keyword.

3.1 · What runs when — one query, four calls



the predicate count after each step is the sample's measured output · ToList is the only call whose result is kept · colours only separate the actors (indigo) from the summary note (amber)

3.2 · Reusing a query — Java Streams vs LINQ

Java — for comparison — the habit you bring

```java
// a Stream is single-use: reuse throws
int[] calls = {0};  // lambdas capture effectively final
Stream<Quote> accepted = quotes.stream()
    .filter(q -> { calls[0]++; return q.isAccepted(); });

long count = accepted.count();      // runs the pipeline
accepted.findFirst();
// IllegalStateException: stream has already
// been operated upon or closed

// to read results more than once, collect first
List<Quote> list = quotes.stream()
    .filter(Quote::isAccepted)
    .toList();
```

C# — Program.cs · #region deferred

```csharp
var calls = 0;
bool Counted(Quote q) { calls++; return q.IsAccepted; }

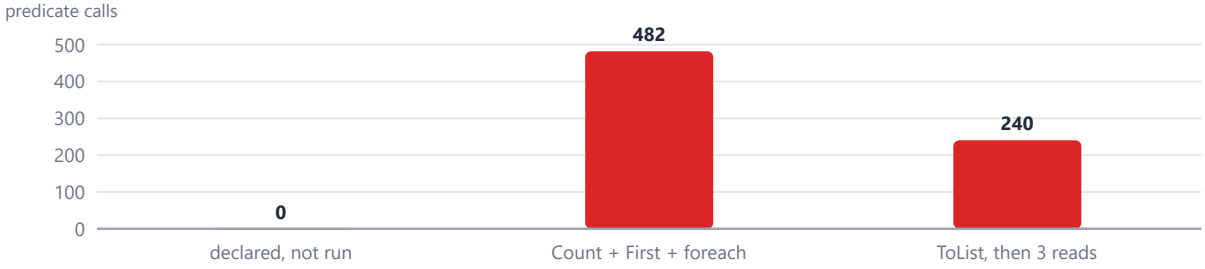
var accepted = quotes.Where(Counted); // method group -> Func
Console.WriteLine($"  declared          calls = {calls}");

var count = accepted.Count();       // pass 1: all quotes
var first = accepted.First();       // pass 2: stops early
foreach (var item in accepted) { } // pass 3: all again
Console.WriteLine($"  three passes      calls = {calls}");

calls = 0;
var list = accepted.ToList();       // one pass, then data
_ = (list.Count, list[0]);
foreach (var item in list) { }
Console.WriteLine($"  ToList + 3 reads  calls = {calls}");
```

Java fails loudly on reuse; C# re-runs quietly. Both pipelines are lazy, but a LINQ query is an `IEnumerable<T>` that starts over on every enumeration. The C# panel printed `calls = 0`, then `calls = 482`, then `calls = 240`. Against a database or a remote API, "quietly re-runs" means three round trips. Analyzer rule CA1851 flags possible multiple enumeration, but it is off by default (CA1851).

3.3 · Predicate calls for the same three reads



calls counted by the sample over the 240-quote book · measured from a real run

Materialising turns three passes into one. 482 = 240 for `Count` + 2 for `First`, which stops at Q-0002, + 240 for the `foreach`. • Measured from `L04.Collections`, and asserted by a test in `L04.QuoteData.Tests`.

3.4 · Deferred means late: captured variables and changing sources

C# · captured — Program.cs · #region captured

```
var threshold = 4_000m;
var pricey = quotes.Where(q => q.Total.Amount > threshold);
Console.WriteLine($"  > threshold, run 1  {pricey.Count()}");

threshold = 6_000m; // the lambda captured the VARIABLE
Console.WriteLine($"  > threshold, run 2  {pricey.Count()}");
```

C# · changed list — Program.cs · #region modified

```
var queue = new List<string> { "Q-0001", "Q-0002" };
try
{
    foreach (var id in queue)
        if (id == "Q-0001") queue.Add("Q-0003");
}
catch (InvalidOperationException ex)
{
    Console.WriteLine($"  {ex.GetType().Name}");
}
```

The lambda captured the variable `threshold`, not its value. The same query printed 123, then 67 after the assignment. On the right, `Add` during enumeration prints `InvalidOperationException`: `List<T>` enumerators check a version number, like Java's fail-fast iterators. Iterate a copy (`queue.ToList()`) when the loop must change the source.

3.5 · Writing a lazy sequence — Kotlin sequence {} vs C# yield return

Kotlin — for comparison — sequence builder

```
fun renewals(inception: LocalDate) = sequence {
    println("  (sequence body starts)")
    var d = inception
    while (true) {
        yield(d)
        d = d.plusYears(1)
    }
}

val renewals = renewals(LocalDate.of(2026, 11, 1))
println("  sequence created")
renewals.take(3).forEach { println("  renewal $it") }
```

C# — Program.cs · #region yield

```
var renewals = Renewals(new DateOnly(2026, 11, 1));
Console.WriteLine("  iterator created");
foreach (var date in renewals.Take(3))
    Console.WriteLine($"  renewal {date:yyyy-MM-dd}");

static IEnumerable<DateOnly> Renewals(DateOnly inception)
{
    Console.WriteLine("  (iterator body starts)");
    for (var d = inception; ; d = d.AddYears(1))
        yield return d;
}
```

Same model, different keyword. Both produce an infinite, lazy sequence of renewal dates, and `Take(3)` stops it after three. The C# run printed `iterator created` *before* `(iterator body starts)`: calling `Renewals` only builds the state machine. A consequence to remember — argument checks at the top of an iterator method also run late, on the first `MoveNext`, not at the call.

4 · The operator map — Streams, Kotlin and TypeScript in LINQ

Most of LINQ is the pipeline you know under SQL-flavoured names. `filter` is `Where`, `map` is `Select`, `flatMap` is `SelectMany`, `reduce`/`fold` is `Aggregate`. The query syntax (`from q in quotes where … select …`) compiles to the same method calls, and it covers only a handful of operators — everything else must be written as a method call (Standard query operators). Most C# teams write method syntax and keep query syntax for joins and nested `from` clauses; 6.2 shows the same left join in both forms.

Most operators added since .NET 6 are ones Kotlin had first — and code you review may predate them. .NET 6 added `Chunk`, `DistinctBy`, `MinBy` and `MaxBy` (What's new in .NET 6). .NET 9 added `CountBy` and `AggregateBy`, which aggregate by key without allocating the groups `GroupBy` builds, and `Index` (What's new in .NET 9 libraries); like every LINQ operator that returns a sequence they are deferred. .NET 10 added `LeftJoin` and `RightJoin` — SQL's joins, not Kotlin's — replacing the `GroupJoin` + `DefaultIfEmpty` idiom; neither has query syntax (Join operations). It also added `Shuffle` and `InfiniteSequence` (Enumerable.Shuffle · Enumerable.InfiniteSequence) and ships `System.Linq.AsyncEnumerable` in the platform, replacing the community `System.Linq.Async` package (System.Linq.AsyncEnumerable in .NET 10) — lesson 05 (Async, Tasks & Concurrency).

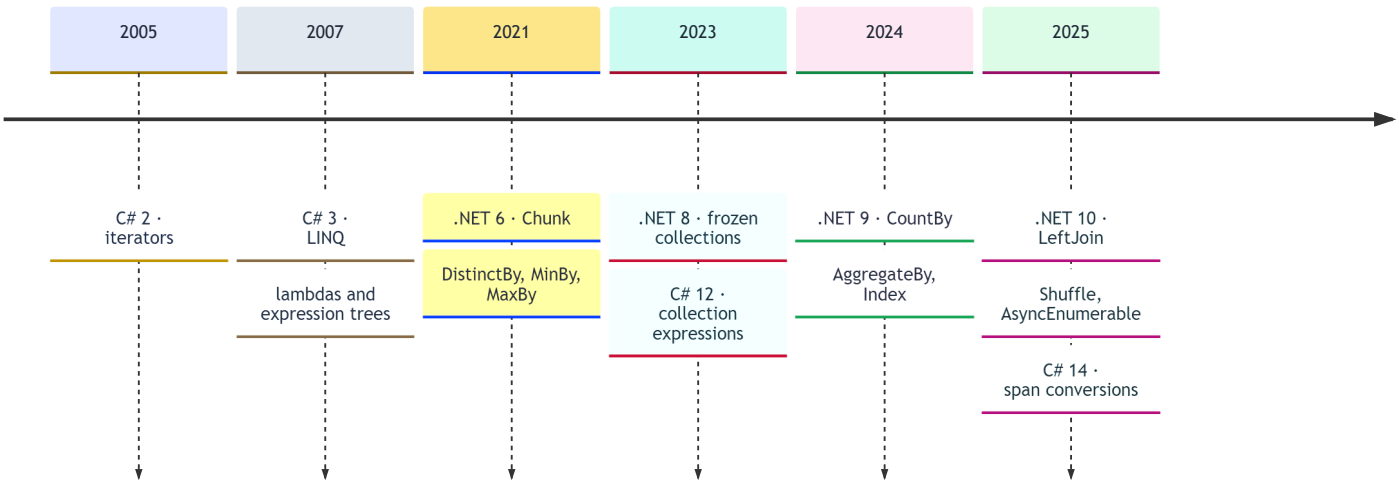
Know which operators execute now and which fail on a missing key. `ToLookup` runs immediately and `GroupBy` is deferred — the test `GroupBy_is_deferred_and_ToLookup_is_immediate` counts the key selector at 0, 240 and 720. A lookup returns an empty group for a key it has never seen; a dictionary indexer throws.

4.1 · Concept map — pipelines you know → LINQ

You already know	In .NET	Kind	Watch for
`filter` · `arr.filter`	`Where`	• Renamed	Deferred and streaming
`map` · `arr.map`	`Select`	• Renamed	`Select((x, i) => …)` passes the index

You already know	In .NET	Kind	Watch for
`flatMap`	`SelectMany`	• Renamed	Also a second `from` in query syntax
`reduce` / `fold`	`Aggregate`	• Renamed	Without a seed it throws on an empty sequence
`Collectors.groupingBy` · Kotlin `groupBy`	`GroupBy` / `ToLookup`	• Trap	`GroupBy` is deferred and regroups on every enumeration; Kotlin's `groupBy` returns a map immediately — that is `ToLookup`
`groupingBy(counting())` · Kotlin `eachCount()`	`CountBy` (.NET 9)	• Trap	Deferred: it recounts on every enumeration and yields pairs, not a map — add `ToDictionary()` where Kotlin gave you a `Map`; no row for an absent key
Kotlin `groupingBy { }.fold`	`AggregateBy` (.NET 9)	• Trap	Deferred like `CountBy`: one accumulator per key, no intermediate groups, rebuilt on every enumeration; Kotlin's `fold` returns a `Map` at once
Kotlin `withIndex()` · `arr.entries()`	`Index()` (.NET 9)	• Renamed	Yields `(Index, Item)` tuples
Kotlin `chunked(n)`	`Chunk(n)` (.NET 6)	• Renamed	Each chunk is an array; the last one holds the remainder
Kotlin `distinctBy`	`DistinctBy` (.NET 6)	• Same idea	Compares by a key without sorting; older code emulates it with `GroupBy` + `First`
Kotlin `maxByOrNull`	`MaxBy` (.NET 6)	• Trap	On an empty sequence `MaxBy` returns null only for reference types; for structs (including `CountBy`'s `KeyValuePair` rows) it throws `InvalidOperationException`
Kotlin `zip` · Guava `Streams.zip`	`Zip`	• Renamed	Stops at the shorter sequence; the overload without a result selector yields tuples
`Collectors.toMap` · Kotlin `associateBy`	`ToDictionary`	• Trap	Duplicate keys throw `ArgumentException`, as `toMap` throws `IllegalStateException`; Kotlin's `associateBy` keeps the last one
`map.get(key)` returning null	`dict[key]` · `lookup[key]`	• Trap	The dictionary indexer throws `KeyNotFoundException`; a lookup returns an empty group
SQL `LEFT JOIN`	`LeftJoin` (.NET 10)	• Different	Before .NET 10: `GroupJoin` + `SelectMany` + `DefaultIfEmpty`
Stream single use	Re-enumerable `IEnumerable<T>`	• Trap	Every enumeration re-runs the whole pipeline (3.2)
`toList()` · spread into an array	`ToList()` · `ToArray()`	• Same idea	The point where the query actually runs
Kotlin `sequence { yield() }`	`yield return`	• Renamed	The method body starts on the first `MoveNext`
`parallelStream()`	PLINQ `AsParallel()`	• Different	`AsParallel()` partitions an *in-memory* sequence across the thread pool, and `AsOrdered()` buys the original order back at a cost. This track does not cover PLINQ; lesson 05 (Async, Tasks & Concurrency) covers Task-based parallelism (`Task.WhenAll`, `Parallel.ForEachAsync`), which is what I/O-bound fan-out needs instead

4.2 · When each feature arrived — dating the LINQ in a codebase



language features and library operators by release year · C# 12 shipped with .NET 8 and C# 14 with .NET 10 · colours only separate the releases · verified against the C# history, the .NET what's-new pages and the API reference

4.3 · AggregateBy — a fold per key without building groups

C# `samples/L04.QuoteData/QuoteDashboard.cs · #region net9-additions`

```
// accepted premium per coverage class in one pass: no intermediate IGrouping objects
public static IEnumerable<KeyValuePair<CoverageClass, decimal>> PremiumByClass(
    IEnumerable<Quote> quotes) =>
    quotes
        .Where(q => q.IsAccepted)
        .AggregateBy(
            keySelector: q => q.Coverage,
            seed: 0m,
            (total, q) => total + q.Total.Amount)
        .OrderBy(kv => kv.Key);
```

This is Kotlin's `groupingBy { }.fold`. One pass keeps one running total per coverage class; `GroupBy` would first collect every quote into a group and then sum each group. The dashboard prints `Class1 136,262` … `Class3 130,512` THB (the canonical tariff, illustrative). Like `CountBy` it returns a deferred sequence of `KeyValuePairs` that is rebuilt on every enumeration (the test `CountBy_and_AggregateBy_are_deferred…` counts it), so sort explicitly when order matters and add `ToList()` or `ToDictionary()` to keep the result.

4.4 · A key that is not there — lookup vs dictionary

C# · missing key `Program.cs · #region missing-key`

```
var byMake = quotes.ToLookup(q => q.Vehicle.Make);
var soldByMake = quotes.CountBy(q => q.Vehicle.Make)
    .ToDictionary();

var teslas = byMake["Tesla"].Count();
Console.WriteLine($"  lookup     {teslas} Tesla quotes");
try { _ = soldByMake["Tesla"]; }
catch (KeyNotFoundException ex)
{
    Console.WriteLine($"  dictionary {ex.GetType().Name}");
}
```

Output · part 9 `L04.Collections, captured`

```
9. a key that is not there
  lookup     0 Tesla quotes
  dictionary KeyNotFoundException
```

Two grouped structures, two answers to a missing key. The `ILookup` from `ToLookup` returns an empty sequence for `Tesla`; the dictionary built from `CountBy` throws `KeyNotFoundException`. Kotlin and Java return null from `map[key]`/`get`, so ported code that null-checks the result crashes instead. Use `TryGetValue` or `GetValueOrDefault`.

5 · Functional C# — delegates, closures and immutable records

A C# lambda becomes a delegate, and delegate types are nominal, not structural. In Kotlin and TypeScript any function with the right shape fits a function type. In C#, `Func<Quote, bool>` and `Predicate<Quote>` have the same shape and still do not convert — the compiler reports CS0029, and the sample converts through `.Invoke` instead. Use `Func` and `Action`

everywhere, and declare a named delegate only where the name documents intent, as `Loading` does below (Lambda expressions).

Rules as values is the functional shape of a rating engine. Each loading rule is a named function in a list, and the total loading is a fold of those rules into one number, which the base premium is then multiplied by: no mutable state, same input, same output. That is what makes the rules trivially testable — the young-driver theory asserts a pure function — and easy to reorder or extend. Because the loadings are *added* before they are applied, the order of the list cannot change the answer, which is a property worth having in a tariff.

Closures capture variables, not values — and C#'s two loops differ. A `for` loop has one loop variable shared by every lambda created in it, so three lambdas print `3,3,3`; a `foreach` gets a fresh variable per iteration and prints `0,1,2`. TypeScript's `for (let …)` creates a fresh binding per iteration, so the TypeScript habit is exactly wrong for a C# `for` loop. A `static` lambda cannot capture at all, which makes the intent explicit.

Immutability comes from records and `with` — shallow, like Kotlin's `copy()`. `with` copies the reference to a `List<string>`, so the renewal and the draft share one list. A record meant to be a value can hold an `ImmutableArray<T>` or another immutable collection, which stops the mutation — but not the surprise in equality: record equality compares an `ImmutableArray<T>` by its array reference, so two records with the same elements in separate arrays are *not* equal (the test `A_record_holding_an_ImmutableArray…` proves it). Override `Equals` with `SequenceEqual` when a record must compare by content. Records themselves are lesson 03 (Types, OOP & Generics).

5.1 · Rating rules as a list of functions, folded with Aggregate

C# — samples/L04.QuoteData/Rating.cs · #region rules-as-functions

```
// a rule is a value: it reads the quote and returns the loading it adds to the base premium
public delegate decimal Loading(RatingInput input);

public static readonly IReadOnlyList<(string Name, Loading Rate)> Rules =
[
    ("young driver", r => r.DriverAge < 25 ? 0.20m : 0m),
    ("claims", r => r.Claims switch
    {
        0 => 0m, 1 => 0.10m, 2 => 0.25m,
        _ => throw new InvalidOperationException(DeclineReason), // 3+ is declined, not loaded
    }),
    ("commercial use", r => r.Use is not VehicleUse.Commercial ? 0m
                            : r.EngineCc > 3_000 ? 0.35m : 0.25m),
];

// fold the rules into one loading, then apply it and the no-claim discount to the base premium
public static decimal Net(RatingInput r) =>
    Round(Base(r) * (1m + Rules.Aggregate(0m, (loading, rule) => loading + rule.Rate(r)))
                  * (1m - NoClaimDiscount(r)));

// claim-free years earn the ladder; a single claim resets it to zero
static decimal NoClaimDiscount(RatingInput r) =>
    (r.Claims == 0 ? r.LicenceYears : 0) switch
    {
        0 => 0m, 1 => 0.20m, 2 => 0.25m, 3 => 0.30m, 4 => 0.40m, _ => 0.50m,
    };
```

Read the fold as `rules.fold(0m) { loading, rule -> loading + rule(r) }`. `Loading` is a named delegate type; each tuple pairs a rule name with a lambda. The collection expression builds an `IReadOnlyList` of tuples. Adding a fleet-size rule is one more line, and every rule can be tested alone: `Rating.Rules.Single(r => r.Name == "young driver").Rate`. These are the rules lesson 02 (C# Language Essentials) wrote as `switch` expressions, re-expressed as values — the track's one illustrative tariff, not a real one; three claims in five years is a decline rather than a loading, which section 6 counts as a `QuoteStatus`.

5.2 · Closures in loops — TypeScript let vs C# for

TypeScript — for comparison closures in loops

```typescript
const byLet: Array<() => number> = [];
for (let i = 0; i < 3; i++)
  byLet.push(() => i);     // a fresh i per pass

const byVar: Array<() => number> = [];
for (var j = 0; j < 3; j++)
  byVar.push(() => j);     // ONE j, shared by all

const run = (fs: Array<() => number>) =>
  fs.map(f => f()).join(",");

console.log(run(byLet));   // 0,1,2
console.log(run(byVar));   // 3,3,3
```

C# Program.cs · #region closures

```csharp
var byFor = new List<Func<int>>();
for (var i = 0; i < 3; i++)
    byFor.Add(() => i);            // ONE i, shared by all

var byForeach = new List<Func<int>>();
foreach (var year in new[] { 0, 1, 2 })
    byForeach.Add(() => year);     // a fresh year per pass

Console.WriteLine($"  for      {Run(byFor)}");
Console.WriteLine($"  foreach  {Run(byForeach)}");

static string Run(List<Func<int>> fs) =>
    string.Join(",", fs.Select(f => f()));
```

C#'s `for` behaves like TypeScript's `var`, not like `let`. The C# run printed `for 3,3,3` and `foreach 0,1,2`. This bites in exactly the code you write in fan-out loops: building a list of tasks or callbacks with an index. Copy the loop variable into a local inside the body, or use `foreach` over a range.

5.3 · Delegates are nominal; with copies are shallow

C# Program.cs · #region delegate-types

```csharp
Func<Quote, bool> isAccepted = q => q.IsAccepted;
// Predicate<Quote> p = isAccepted;  // error CS0029
Predicate<Quote> asPredicate = isAccepted.Invoke;

var counted = quotes.Count(isAccepted);
var found = quotes.ToList().FindAll(asPredicate).Count;
Console.WriteLine($"  Count {counted}, FindAll {found}");
```

C# Program.cs · #region with-shallow

```csharp
var draft = new DraftQuote("Q-0001", Notes: ["created"]);
var renewal = draft with { QuoteId = "Q-0002" };
renewal.Notes.Add("renewal");   // the SAME List<string>

var notes = string.Join(" | ", draft.Notes);
var same = ReferenceEquals(draft.Notes, renewal.Notes);
Console.WriteLine($"  draft notes  {notes}");
Console.WriteLine($"  same list?   {same}");
```

Two small surprises for a Kotlin engineer. Left: the commented line does not compile (CS0029, checked on the build SDK); `isAccepted.Invoke` is a method group that converts to any compatible delegate type. Both counts print 127. Right: the run printed `draft notes created | renewal` and `same list? True` — changing the renewal's notes changed the draft's.

6 · Quote analytics — the dashboard habit in LINQ

A dashboard query is group, aggregate, order, project — whether it feeds a BI tool or a LINQ pipeline. You have written a few hundred of them in SQL for a BI dashboard. Written in LINQ they are type-checked, refactorable with the domain model and unit-testable against a fixed dataset. `QuoteBook.Generate()` produces the same 240 MotorQuote quotes on every machine — vehicles, drivers, coverage classes and statuses derived arithmetically from the index — so every number below is reproducible.

Prefer `CountBy` and `AggregateBy` when you need one number per key. `GroupBy` is right when you need the members of each group or several aggregates at once, as the conversion query does (code in 8.1). `CountBy` emits no row for a key that has no items, so a dashboard that must show zero rows walks a fixed list and looks each key up — `BandOrder` in 6.1.

Pin the numbers with a test. `Dashboard_numbers_match_the_captured_lesson_output` asserts the accepted counts, bands, top makes, premium totals and the 21 policies still pending issuance. A changed rating rule fails a test before a chart silently changes — the discipline of a query review, enforced by `dotnet test`.

6.1 · Premium bands — a switch expression inside CountBy

C# samples/L04.QuoteData/QuoteDashboard.cs · #region premium-bands

```csharp
public static readonly string[] BandOrder = ["< 3k", "3k-6k", "6k-9k", "9k-12k", "12k+"];

public static string BandOf(Money premium) => premium.Amount switch
{
    < 3_000m => "< 3k",
    < 6_000m => "3k-6k",
    < 9_000m => "6k-9k",
    < 12_000m => "9k-12k",
    _ => "12k+",
};

// .NET 9 CountBy: one pass, one counter per key. A band with no quotes gets no key,
// so walk BandOrder and look each band up: an empty band shows 0, not a missing row
```

```
public static IReadOnlyList<KeyValuePair<string, int>> Bands(
    IEnumerable<Quote> quotes)
{
    var counts = quotes
        .Where(q => q.Status != QuoteStatus.Declined) // a declined quote has no premium
        .CountBy(q => BandOf(q.Total))
        .ToDictionary();
    return BandOrder
        .Select(band => KeyValuePair.Create(band, counts.GetValueOrDefault(band)))
        .ToList();
}
```

The band is a pure function of the premium, so counting stays one line. `BandOf` uses relational patterns from lesson 02 (C# Language Essentials); `CountBy` counts per band in one pass, and `ToDictionary()` keeps the counts. A band with no quotes has no key, so `Bands` walks `BandOrder` and `GetValueOrDefault` supplies the 0 — the test `A_band_without_quotes_still_gets_a_row_with_zero` proves it. The method returns a materialised list, the rule from 2.5.

6.2 · Issuance — LeftJoin, and the query-syntax left join it replaces

C# · LeftJoin, .NET 10 — QuoteDashboard.cs · #region left-join

```
const string Unset = "(pending)"; // no policy issued yet

// .NET 10: accepted quotes, with a policy if one exists
public static IEnumerable<Issued> Issuance(
    IEnumerable<Quote> quotes,
    IEnumerable<Policy> policies) =>
    quotes
        .Where(q => q.IsAccepted)
        .LeftJoin(policies,
            q => q.QuoteId, p => p.QuoteId,
            (q, p) => (q.QuoteId, p?.PolicyNumber ?? Unset));
```

C# · query syntax — QuoteDashboard.cs · #region left-join-query

```
// before .NET 10: the same left join in query syntax
public static IEnumerable<Issued> IssuanceQuery(
    IEnumerable<Quote> quotes,
    IEnumerable<Policy> policies) =>
    from q in quotes
    where q.IsAccepted
    join p in policies on q.QuoteId equals p.QuoteId
        into issued
    from p in issued.DefaultIfEmpty()
    select (q.QuoteId, p?.PolicyNumber ?? Unset);
```

Both keep every accepted quote; a missing policy arrives as `null`. On the left, `LeftJoin` says it in one call. On the right, `join … into issued` followed by `from p in issued.DefaultIfEmpty()` is the query-syntax shape of a left join — the `GroupJoin` + `DefaultIfEmpty` idiom you will meet in code written before .NET 10 (Join operations). The selector's second parameter is nullable either way, so `p?.PolicyNumber ?? Unset` is required, not defensive. The run reports 127 accepted and 21 pending, and the test `The_C_sharp_query_syntax_left_join_matches_LeftJoin` asserts the two return the same sequence.

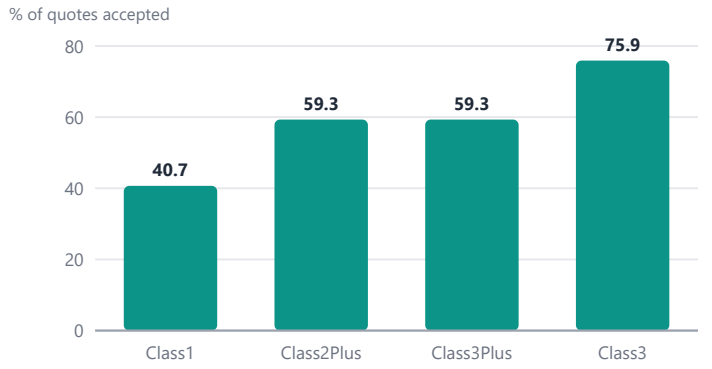
6.3 · What the dashboard prints

Output — L04.QuoteAnalytics, after the conversion and band lines charted in 6.4–6.5 — captured on the build machine

```
top makes, accepted quotes
  1. Toyota    27
  2. Honda     22
  3. MG        16
  4. Isuzu     15
  5. Nissan    14
accepted premium by class, THB
  Class1          136,262
  Class2Plus      169,483
  Class3Plus      174,299
  Class3          130,512
issuance       127 accepted, 21 pending
export batches 100 + 100 + 40
lookup         49 Toyota quotes
```

Every figure here except the batch sizes and the lookup count is asserted by a test. The premiums come from the track's canonical tariff, which is illustrative. The ranking is Toyota 27, Honda 22, MG 16, Isuzu 15 and Nissan 14; ties are still possible on another dataset, so the query adds `ThenBy(kv => kv.Key)` to order them by name (code in 8.2). `Chunk(100)` split the book into export batches of 100 + 100 + 40.

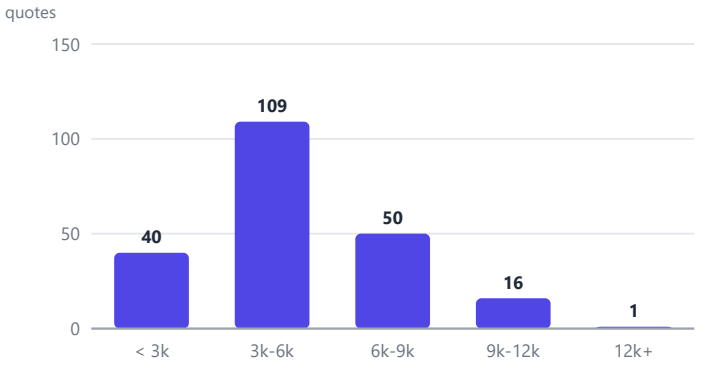
6.4 · Conversion rate by coverage class



accepted ÷ quoted per class · the query is shown in 8.1 · measured from the sample's dashboard output

Acceptance rises as cover gets cheaper — 40.7% for Class 1 to 75.9% for Class 3. The dataset builds that gradient in on purpose, so the query has something to find: 127 of 216 quotes accepted overall. • **Measured**, illustrative data.

6.5 · Quotes per premium band



the 216 priced quotes by total premium band, THB · the 24 declined quotes have no premium to band · measured from the sample's dashboard output

The 6k–9k band holds 50 quotes; < 3k holds 40. Band edges are business choices, so the chart is only as meaningful as `BandOf`. The bars sum to 216, not 240 — the test `Bands_account_for_every_priced_quote` checks that no *priced* quote falls outside a band. • **Measured**, illustrative data.

7 · Expression trees — code as data, and a DynamoDB query generator

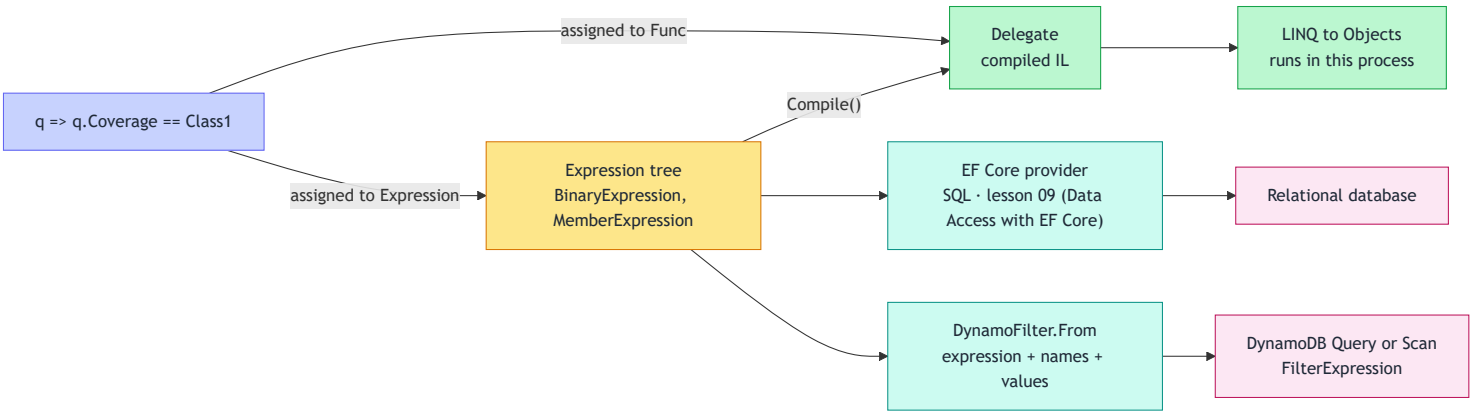
The same lambda compiles either to executable code or to a data structure, depending only on the type it is assigned to. Assigned to `Func<Quote, bool>` it becomes a delegate. Assigned to `Expression<Func<Quote, bool>>` the compiler emits code that builds a tree of nodes describing it (Expression trees). `IQueryable<T>` operators take expressions, which is how EF Core turns a C# predicate into SQL — lesson 09 (Data Access with EF Core).

A query generator is a translator over that tree. You built a DynamoDB query auto-generator before; in C# the compiler hands you the parsed predicate, and the library is one recursive switch over node shapes. `&&`, `||` and `!` become `AND`, `OR` and `NOT`; comparisons become comparators; `StartsWith` becomes `begins_with`; `Contains` becomes `contains` or `IN`. Every value goes through a `:value` placeholder, which DynamoDB always requires; every attribute goes through a `#name` placeholder, which DynamoDB requires only for reserved words and names with special characters — using it everywhere keeps the generator safe (DynamoDB expression attribute names · DynamoDB condition and filter expressions).

Compile time accepts what translation rejects. Any expression lambda that type-checks compiles, but a translator — yours or EF Core's — supports a subset. The computed property `IsAccepted` and a call to `ToUpperInvariant` both compile and both fail at translation, each with its own exception: `DynamoFilter` throws `NotSupportedException` naming the node it met, while EF Core throws `InvalidOperationException` ("The LINQ expression … could not be translated") — lesson 09 (Data Access with EF Core) pins that type in a test. The compiler itself refuses statement lambdas, `?.`, switch expressions, collection expressions, tuple literals and `is` patterns inside an expression tree; an interpolated string compiles, to a `string.Format` call that few providers can translate.

C# 14 changed what `array.Contains(x)` binds to inside an expression. With the new implicit span conversions it binds to `MemoryExtensions.Contains`, a `ReadOnlySpan<T>` method (7.4), and such trees throw when compiled with interpretation (C# 14 overload resolution with span parameters). The translator unwraps the conversion; for a provider you do not own, call `Enumerable.Contains` explicitly or cast to `IEnumerable<T>`.

7.1 · One lambda, two destinations



indigo = your source · green = code · amber = data the compiler builds · teal = a translator · rose = outside this process

7.2 · Func<T,bool> or Expression<Func<T,bool>>?

Func<T, bool> — a delegate

- Compiled IL; call it directly
- Used by `Enumerable` (LINQ to Objects)
- Any statement lambda, `?.`, switch expression or local function is allowed
- Opaque to a provider: it cannot be translated to SQL or a filter
- Choose it for in-memory rules — the rating rules in 5.1

Expression<Func<T, bool>> — a tree

- Data: nodes you can walk, rewrite or translate
- Used by `Queryable` — EF Core, your own generator
- Expression lambdas only; newer syntax is refused by the compiler
- Every provider supports a subset — unsupported calls fail at run time
- Choose it when the predicate must run somewhere else: SQL, DynamoDB, a search index

7.3 · Func vs Expression, a span surprise, and a generated filter

C# — samples/L04.QuoteAnalytics/Program.cs · #region expression-demo

```
Func<Quote, bool> compiled = q => q.Coverage == CoverageClass.Class1;
Expression<Func<Quote, bool>> tree = q => q.Coverage == CoverageClass.Class1;
Console.WriteLine($"compiled  {compiled.GetType().Name}  -> {compiled(quotes[0])}");
Console.WriteLine($"tree      {tree.Body.NodeType}  -> {tree.Body}");

string[] shortlist = ["Toyota", "Honda"];
Expression<Func<Quote, bool>> inList = q => shortlist.Contains(q.Vehicle.Make);
var call = (MethodCallExpression)inList.Body;
Console.WriteLine($"binds to  {call.Method.DeclaringType!.Name}.{call.Method.Name}");

var minPremium = 15_000m;
var filter = DynamoFilter.From<Quote>(q =>
    q.Coverage == CoverageClass.Class1
    && q.Total.Amount >= minPremium
    && q.Vehicle.Make.StartsWith("To"));
Console.WriteLine($"FilterExpression  {filter.Expression}");
Console.WriteLine($"Names             {string.Join(", ", filter.Names)}");
Console.WriteLine($"Values            {string.Join(", ", filter.Values)}");
```

> **Three experiments in one region.** The first pair declares the same lambda twice. The second shows what C# 14 binds `shortlist.Contains` to. The third hands a predicate with a captured `minPremium` to the generator. The output follows in 7.4.

7.4 · What the expression experiments print

Output — L04.QuoteAnalytics (expression part) captured on the build machine

```
compiled  Func`2  -> True
tree      Equal  -> (Convert(q.Coverage, Int32) == 0)
binds to  MemoryExtensions.Contains
FilterExpression  ((#n0 = :v0 AND #n1.#n2 >= :v1) AND begins_with(#n3.#n4, :v2))
Names             [#n0, Coverage], [#n1, Total], [#n2, Amount], [#n3, Vehicle], [#n4, Make]
Values            [:v0, Class1], [:v1, 15000], [:v2, To]
```

Read the second line slowly. The tree shows the enum comparison as `Convert(q.Coverage, Int32) == 0` — the compiler lowered it, which is why the translator strips `Convert` nodes and maps the number back to `Class1`. The captured `minPremium` arrives as the value 15000, evaluated when the filter was built. Five name placeholders cover one top-level attribute and two nested paths.

7.5 · The translator — pattern matching is the visitor

C# samples/L04.QueryGen/DynamoFilter.cs · #region translate

```
// one switch over node shapes: pattern matching is the visitor
public string Condition(Expression e) => e switch
{
    BinaryExpression { NodeType: ExpressionType.AndAlso } b
        => $"({Condition(b.Left)} AND {Condition(b.Right)})",
    BinaryExpression { NodeType: ExpressionType.OrElse } b
        => $"({Condition(b.Left)} OR {Condition(b.Right)})",
    UnaryExpression { NodeType: ExpressionType.Not } u
        => $"NOT ({Condition(u.Operand)})",
    BinaryExpression b when Comparators.TryGetValue(b.NodeType, out var op)
        => Compare(b.Left, op, b.Right),
    MethodCallExpression { Method.Name: "StartsWith", Object: { } s } m when IsOnItem(s)
        => $"begins_with({Path(s)}, {Operand(m.Arguments[0], typeof(string))})",
    MethodCallExpression { Method.Name: "Contains" } m
        => Contains(m),
    MemberExpression m when m.Type == typeof(bool) && IsOnItem(m)
        => $"{Path(m)} = {Constant(true)}",
    _ => throw Unsupported(e),
};
```

One switch expression replaces a visitor class hierarchy. Property patterns match node shapes (`BinaryExpression { NodeType: ExpressionType.AndAlso }`), and anything unmatched throws. `DynamoFilter.From<T>` calls `Condition(predicate.Body)` and returns the text with its name and value maps. Bare boolean members become `= :v`, because DynamoDB cannot use a Boolean attribute on its own as a condition (DynamoDB condition and filter expressions). A member counts as a stored attribute when it is a field or an auto-property, so a hand-written getter such as `IsAccepted` is refused as computed C#.

A filter expression is not a query plan. DynamoDB applies a filter after `Query` has read the items, so the read capacity consumed is the same with or without it, and the 1 MB page limit applies before filtering (DynamoDB Query filter expressions). Generate filters for the residual conditions; design partition and sort keys for the access pattern.

7.6 · What the generator translates — each row is a passing test

C# predicate	Filter expression	Result
`q.QuoteId == "Q-0001"`	`#n0 = :v0`	✓ translated
`r.Year < 2020` … `>=`, `!=`	`< <= > >= = <>`	✓ translated
`2020 < r.Year`	`#n0 > :v0` — constant moved right	✓ translated
`(a || b) && !(c)`	`((… OR …) AND NOT (…))`	✓ translated
`q.Vehicle.Make.StartsWith("To")`	`begins_with(#n0.#n1, :v0)`	✓ translated
`q.Vehicle.Model.Contains("Revo")`	`contains(#n0.#n1, :v0)`	✓ translated
`makes.Contains(q.Vehicle.Make)`	`#n0.#n1 IN (:v0, :v1)`	✓ translated
`makes.Contains(…)` with 0 or 101 values	— (DynamoDB: 1 to 100 values)	✗ refused
`q.Coverage == CoverageClass.Class3Plus`	value `Class3Plus`, not 2	✓ translated
`r.Start >= from` (a `DateOnly`)	value `2026-11-01`	✓ translated
`s.Make == "Toyota"` (a get-only auto-property)	`#n0 = :v0`	✓ translated
a captured local changed after `From`	keeps the value read at translation	by design
`q.IsAccepted` (computed property)	—	✗ refused
`Make.ToUpperInvariant() == "TOYOTA"`	—	✗ refused
`q.Vehicle.Make.EndsWith("a")`	— (DynamoDB has no `ends_with`)	✗ refused
`r.Tags.Contains("fleet")` (a list attribute)	— (DynamoDB `contains` supports lists; this translator does not map it)	✗ refused

`q` is a `Quote`; `r` is the test file's own `Row(Id, Renewal, Year, Start, Tags)` record and `s` its `Stored` class. Every row is a test in `DynamoFilterTests.cs`; ✗ **refused** means the translator throws `NotSupportedException`.

7.7 · Testing the generator, including what it refuses

C# — samples/L04.QueryGen.Tests/DynamoFilterTests.cs · #region tests

```
[Fact]
public void Captured_variables_are_read_when_the_filter_is_built()
{
    var min = 10_000m;
    Expression<Func<Quote, bool>> predicate = q => q.Total.Amount >= min;

    var built = DynamoFilter.From(predicate);
    min = 99_999m; // too late for `built`: it was translated already

    Assert.Equal("#n0.#n1 >= :v0", built.Expression);
    Assert.Equal(10_000m, built.Values[":v0"]);
    Assert.Equal(99_999m, DynamoFilter.From(predicate).Values[":v0"]);
}

[Fact]
public void A_computed_property_compiles_but_cannot_be_translated()
{
    var ex = Assert.Throws<NotSupportedException>(
        () => DynamoFilter.From<Quote>(q => q.IsAccepted));

    Assert.Contains("IsAccepted", ex.Message);
}
```

Test the translator's contract, not DynamoDB. No AWS SDK and no table: the tests compare expression text and placeholder maps. The first test here pins a subtle rule — a captured variable is read when `From` runs, so a filter built earlier keeps the old value. The second proves a predicate can compile and still be untranslatable. The project holds 24 test cases, covering every row of 7.6. • **Measured** .

8 · Visual Basic LINQ — the richer query syntax

Visual Basic spells more of LINQ as clauses than C# does. The C# language reference lists eight clause keywords — `from`, `where`, `select`, `group`, `into`, `orderby`, `join`, `let` — plus six contextual words that only work inside them (`in`, `on`, `equals`, `by`,

`ascending`, `descending`) (C# query keywords). Visual Basic has fourteen query clauses; six of them have no C# query keyword: `Aggregate`, `Distinct`, `Skip`, `Skip While`, `Take` and `Take While` (Introduction to LINQ in Visual Basic). A VB estate's queries read like SQL, and many never call a LINQ method by name.

Underneath it is the same LINQ. The clauses compile to the same `Enumerable` calls, deferred and immediate execution work the same way, and the VB console prints the same conversion and top-makes figures as the C# dashboard. An `Aggregate` clause that *begins* a query runs immediately and returns one value, or one object with a property per aggregate function; inside a `From` query it adds per-row aggregates and the query stays deferred (Aggregate clause).

Porting VB queries to C# is mechanical but not keyword-for-keyword. `Group By … Into Quoted = Count()` becomes `GroupBy` plus a `Select` (8.1); `Aggregate … Into`, `Distinct`, `Skip While` and `Take While` become method calls (8.3, 8.4); anonymous types become records or tuples when they cross a method boundary. Plan the port of a whole VB estate with lesson 06 (VB.NET for Legacy Estates).

8.1 · Conversion by class — C# methods vs VB Group By … Into

C# · methods — QuoteDashboard.cs · #region conversion

```
public static IReadOnlyList<ClassConversion> Conversion(
    IEnumerable<Quote> quotes) =>
    quotes
        .Where(q => q.Status != QuoteStatus.Declined)
        .GroupBy(q => q.Coverage)
        .OrderBy(g => g.Key)
        .Select(g => new ClassConversion(
            g.Key,
            Quoted: g.Count(),
            Accepted: g.Count(q => q.IsAccepted)))
        .ToList();
```

VB · Group By — Program.vb · #region conversion

```
Dim conversion =
    From q In quotes
    Where q.Status <> QuoteStatus.Declined
    Group By q.Coverage Into
        Quoted = Count(),
        Accepted = Count(q.IsAccepted)
    Order By Coverage
```

VB names the aggregates inside the grouping clause. `Count(q.IsAccepted)` is VB's conditional count; C# writes `g.Count(q => q.IsAccepted)` inside a `Select`. The VB query has no `ToList`, so it stays deferred until the `For Each` that prints it — the C# method materialises before returning.

8.2 · Top makes — CountBy and Index vs a five-line VB query

C# · CountBy — QuoteDashboard.cs · #region top-makes

```
public static IEnumerable<MakeRank> TopMakes(
    IEnumerable<Quote> quotes, int take = 5) =>
    quotes
        .Where(q => q.IsAccepted)
        .CountBy(q => q.Vehicle.Make)
        .OrderByDescending(kv => kv.Value)
        .ThenBy(kv => kv.Key)
        .Take(take)
        .Index()
        .Select(x => new MakeRank(
            Rank: x.Index + 1, x.Item.Key, x.Item.Value));
```

VB · Group By — Program.vb · #region top-makes

```
Dim topMakes = From q In quotes
               Where q.IsAccepted
               Group By q.Vehicle.Make Into Sold = Count()
               Order By Sold Descending, Make
               Take 5
```

The same ranking, two idioms. C# uses the .NET 9 `CountBy` and `Index` to count and number in the pipeline; VB groups, orders by the named aggregate and `Take 5`, and numbers the rows in the loop that prints them. Both print Toyota 27, Honda 22, MG 16, Isuzu 15 and Nissan 14.

8.3 · Aggregate … Into — a clause C# does not have

C# · port — QuoteDashboard.cs · #region summary

```
// C# port of Aggregate: one pass, three reads
public static Summary Summarise(IEnumerable<Quote> quotes)
{
    var amounts = quotes
        .Where(q => q.IsAccepted)
        .Select(q => q.Total.Amount)
        .ToList();
    return (amounts.Count, amounts.Sum(), amounts.Max());
}
```

VB · Aggregate — Program.vb · #region aggregate

```
Dim book = Aggregate q In quotes
           Where q.IsAccepted
           Into Policies = Count(),
                Premium = Sum(q.Total.Amount),
                Largest = Max(q.Total.Amount)
```

VB's `Aggregate` runs immediately and returns one object. It returned `Policies`, `Premium` and `Largest` properties: `127 quotes, 610,555 THB, largest 12,182`. C# has no such clause: the port materialises the amounts once with `ToList` and reads `Count`, `Sum` and `Max` from the list.

8.4 · Distinct, Skip While and Take While — clauses C# spells as methods

C# · port — QuoteDashboard.cs · #region window

```csharp
// C# port of Distinct, Skip While and Take While
public static IEnumerable<string> Makes(
    IEnumerable<Quote> quotes) =>
    quotes.Select(q => q.Vehicle.Make).Distinct().Order();

public static IEnumerable<Priced> Window(
    IEnumerable<Quote> quotes, decimal lo, decimal hi) =>
    quotes
        .OrderBy(q => q.Total.Amount)
        .SkipWhile(q => q.Total.Amount < lo)
        .TakeWhile(q => q.Total.Amount < hi)
        .Select(q => (q.QuoteId, q.Total.Amount));
```

VB · Skip While — Program.vb · #region distinct-skip-while

```vb
Dim makes = From q In quotes
            Select q.Vehicle.Make
            Distinct
            Order By Make

Dim window = From q In quotes
             Order By q.Total.Amount
             Skip While q.Total.Amount < 8600D
             Take While q.Total.Amount < 8900D
             Select q.QuoteId, q.Total.Amount
```

Both cut the same window from the premium-ordered book. `OrderBy` is stable, so the two quotes tied at 8,685.55 keep their book order (Q-0139, then Q-0219) in C# and in VB. `Distinct` and `Order` give the eight makes. The test `The_C_sharp_ports_reproduce_the_captured_Visual_Basic_output` asserts both ports against the figures the VB console printed.

8.5 · What the VB console prints

Output — L04.VbLinq, after the conversion and top-makes lines — captured on the build machine

```
accepted       127 quotes, 610,555 THB, largest 12,182
makes          BYD,Ford,Honda,Isuzu,Mazda,MG,Nissan,Toyota
premiums from 8,600 to under 8,900 THB
  Q-0139   8,685.55
  Q-0219   8,685.55
  Q-0005   8,701.67
  Q-0015   8,862.81
  Q-0175   8,862.81
```

The conversion and top-makes lines, not repeated here, match the C# dashboard figure for figure: same records, same dataset, a different query language on the same runtime.

8.6 · Which operators have query syntax

Operator	C#	Visual Basic	Syntax
Where · Select · OrderBy	`where` · `select` · `orderby`	`Where` · `Select` · `Order By`	✓ both
GroupBy · Join · Let	`group … by … into` · `join … on … equals` · `let`	`Group By … Into` · `Join … On … Equals` · `Let`	✓ both
Count or Sum per group	`g.Count()` inside `select`	`Into Quoted = Count()`	VB clause
Distinct · Skip · Take	`.Distinct()` · `.Skip(n)` · `.Take(n)`	`Distinct` · `Skip n` · `Take n`	VB clause
SkipWhile · TakeWhile	`.SkipWhile(…)` · `.TakeWhile(…)`	`Skip While` · `Take While`	VB clause
Aggregate a whole source	`quotes.Count(…)`, `.Sum(…)`, `.Max(…)`	`Aggregate q In quotes Into Count(), Sum(…)`	VB clause
CountBy · AggregateBy · LeftJoin	`.CountBy(q => …)`	`.CountBy(Function(q) …)`	methods

9 · Hands-on — build, test and run the lesson-04 samples

7 projects, one command; the 43 tests are the safety net. 24 pin the generator's translation rules and refusals; 19 pin the dataset, the dashboard figures, the C# ports of the VB queries, the pure rating rule, the deferred-versus-immediate behaviour of 3.3 and section 4, and the collection and record claims made in sections 2, 4 and 5 — so a changed rule fails a test before a captured panel here goes stale.

Try three changes, then run the tests. In `GroupBy_is_deferred_and_ToLookup_is_immediate` swap `GroupBy` and `ToLookup` and read which assertion fails; in the `readonly-views` region (2.3) add a second `classes.Add("Class3Plus")` and predict which of the four counts change before you run `L04.Collections`; teach `DynamoFilter` to emit `BETWEEN` for `r.Year >= 2020 && r.Year <= 2024` — DynamoDB has it (DynamoDB condition and filter expressions) — and add the test.

9.1 · Commands

Terminal

```
# build, test and run every lesson-04 project
python 0-script/verify_samples.py --only 4

# or one at a time, from the repository root
dotnet run  --project lesson-04-linq-collections-functional/samples/L04.Collections
dotnet run  --project lesson-04-linq-collections-functional/samples/L04.QuoteAnalytics
dotnet run  --project lesson-04-linq-collections-functional/samples/L04.VbLinq
dotnet test lesson-04-linq-collections-functional/samples/L04.QueryGen.Tests
dotnet test lesson-04-linq-collections-functional/samples/L04.QuoteData.Tests
```

`verify_samples.py` **must end with 7/7 sample projects passed.** The consoles need no input, network or database and exit within seconds.

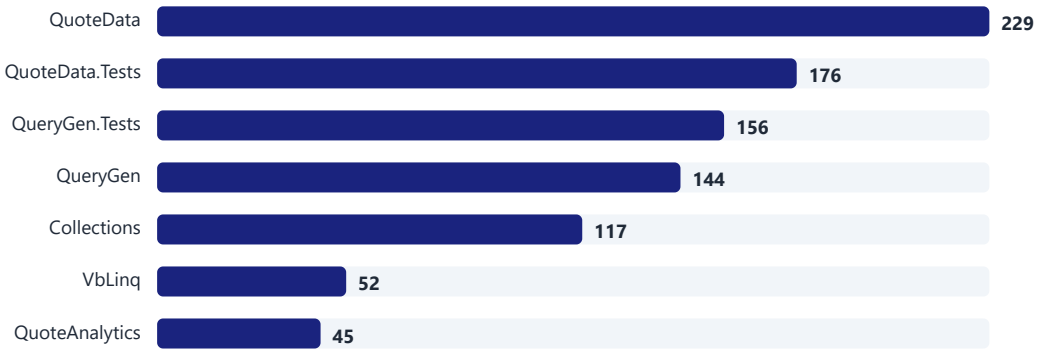
9.2 · What the tests report

Output — dotnet test, durations omitted captured on the build machine

```
Passed!  - Failed: 0, Passed: 24, Skipped: 0, Total: 24 - L04.QueryGen.Tests.dll (net10.0)
Passed!  - Failed: 0, Passed: 19, Skipped: 0, Total: 19 - L04.QuoteData.Tests.dll (net10.0)
```

43 tests pass: 24 in the generator project and 19 in the data project. • Measured from the two test files, and matching this run. If a rating rule or the dataset changes, `Dashboard_numbers_match_the_captured_lesson_output` fails first — re-capture the panels that quote those figures (3.3, 6.3 to 6.5).

9.3 · Where the lesson-04 code lives



non-blank, non-comment lines of C# and VB per sample project · measured when this PDF was built

The largest project is QuoteData (229 lines); the whole VB dashboard console is 52. The two test projects hold 332 lines and 43 test cases against 373 lines in the two libraries they test. • Measured with `loc()`; line count is a size signal, not a quality score.

⚠ Traps

1. **"A pipeline runs once."** A Java stream does; a LINQ query over `IEnumerable<T>` re-runs on every enumeration — 482 predicate calls for three reads in 3.3. Materialise with `ToList` before reading twice, and return materialised results from services.
2. **"Read-only means nobody can change it."** `IReadOnlyList<T>` and `AsReadOnly()` are views: the owner can still add, and a caller holding the `IReadOnlyList<T>` can cast it back to `List<T>` (2.3); the `AsReadOnly()` wrapper blocks that cast but not the owner's changes. Hand out `ImmutableArray<T>` or a frozen copy when that matters — and a record compares an `ImmutableArray<T>` by reference (5).
3. **"Loop variables are captured per iteration."** True for `foreach` and for TypeScript's `let`, false for a C# `for` loop: `3,3,3` in 5.2.
4. **"It compiled, so the provider can run it."** An expression tree type-checks C#, not the provider's subset: computed properties and unknown methods fail at translation (7.6), and in C# 14 `array.Contains` binds to a span method that interpreters reject.
5. **"map[key] returns null."** A `Dictionary` indexer throws `KeyNotFoundException` (4.4) and `ToDictionary` throws on duplicate keys (4.1).

✓ Checkpoint — you are ready for lesson 05 when

1. `python 0-script/verify_samples.py --only 4` reports 7/7 passed on your machine.
2. Given a method that returns `IEnumerable<T>`, you can say whether it is a query or data, and how many times a caller's code will run it.
3. You can choose the parameter and return type of a repository method from table 2.5 and say what each one promises its caller.
4. You can rewrite a Kotlin `groupBy` + `mapValues { it.value.size }` as `CountBy(…).ToDictionary()`, and a `GroupJoin/DefaultIfEmpty` left join as `LeftJoin`.
5. You can extend `DynamoFilter` with a new operator and a test, and say which predicates it must refuse.
6. You can read a VB `Group By … Into` or `Aggregate … Into` query aloud as C#.

🔍 Check yourself

1. A query `quotes.Where(p)` is counted, then its first item is read, then it is looped over. How many times does `p` run for 240 quotes whose second item is the first match, and which of `GroupBy` and `ToLookup` would have run immediately instead?
2. A service returns `classes.AsReadOnly()`. Can a caller see a later change, and can it cast the result back to `List<T>`? What do you return when the caller must not see changes?
3. Why does a `for` loop that adds `() => i` three times print `3,3,3`, and what prints `0,1,2`?
4. What is the difference between `Func<Quote, bool>` and `Expression<Func<Quote, bool>>` for the same lambda, and which one can EF Core translate?
5. Why does a DynamoDB filter expression not reduce the read capacity a `Query` consumes?
6. What does `shortlist.Contains(q.Vehicle.Make)` bind to inside an expression tree under C# 14, and how do you keep a provider working?
7. When does a VB `Aggregate` clause run immediately, and what C# does `Skip While` become?

Lesson 04 in one line: choose collections by the promise they make — read-only is a view, immutable is a guarantee; a LINQ query is a re-runnable recipe, so materialise at boundaries; delegates are nominal and `for` loops share their variable; the same lambda is code for `Enumerable` and data for `Queryable`, which is how you build a query generator; and Visual Basic says more of it in clauses.
Next: lesson 05 (Async, Tasks & Concurrency) — `Task`, `async/await`, cancellation and channels for fan-out to rating partners.